To-Do List App: REST API Planning

Assignment 7 | Backend (Node.js + Express)

1. Introduction

This document identifies and plans the REST APIs needed for a To-Do List application. It covers each endpoint's URL, HTTP method, request body, and expected responses, explains why each API is needed, shows how they together cover CRUD, and notes the challenges expected during implementation.

Base URL: `http://localhost:5000`

Resource: `tasks`

Task data model:

Field	Type	Description
id	Number	Unique identifier, generated by the server
title	String	Name of the task (required)
description	String	Extra details (optional, defaults to empty string)
completed	Boolean	Task status (defaults to false)

2. API Summary

#	Purpose	Method	Endpoint	Success Code

1	Health check	GET	`/`	200
2	Get all tasks	GET	`/tasks`	200
3	Get one task	GET	`/tasks/:id`	200
4	Create a task	POST	`/tasks`	201
5	Update a task	PUT	`/tasks/:id`	200
6	Delete a task	DELETE	`/tasks/:id`	200

3. API Details and Reasoning

3.1 Health Check

- **Endpoint:** `GET /`
- **Request body:** None
- **Success response (200):** `To-Do List API is running`
- **Reasoning:** A simple route to confirm the server is up and reachable before testing anything else. It is useful for debugging and, later, for deployment monitoring.

3.2 Get All Tasks

- **Endpoint:** `GET /tasks`
- **Request body:** None
- **Success response (200):**

[
  { "id": 1, "title": "Learn Express", "description": "First API", "completed": false },

{ "id": 2, "title": "Learn MongoDB", "description": "", "completed": false }
]

- **Empty list (200):** `[ ]`
- **Reasoning:** The app needs to display the full list of tasks on the home screen. GET is used because it only reads data and does not change anything on the server. An empty list is still a valid 200 response, not an error.

3.3 Get One Task by ID

- **Endpoint:** `GET /tasks/:id` (example: `/tasks/1`)
- **Request body:** None
- **Success response (200):**

{ "id": 1, "title": "Learn Express", "description": "First API", "completed": false }

- **Error response (404):**

{ "message": "Task not found" }

- **Reasoning:** Needed to show the details of a single task or to pre-fill an edit form. The ID goes in the URL because it identifies a specific resource. A 404 tells the client clearly that the resource does not exist.

3.4 Create a Task

- **Endpoint:** `POST /tasks`
- **Request body (JSON):**

{ "title": "Learn Express", "description": "Create my first API" }

- `title` is required and must be a non-empty string. `description` is optional.
- **Success response (201):**

{ "id": 1, "title": "Learn Express", "description": "Create my first API", "completed": false }

- **Error response (400):**

{ "message": "Task title is required" }

- **Reasoning:** Users must be able to add new tasks. POST is used because it creates a new resource and is not idempotent (each call creates a new task). The server generates the `id` and sets `completed` to false, so the client cannot send inconsistent values. 201 (Created) is the correct status for a newly created resource, and 400 is returned for invalid input.

3.5 Update a Task

- **Endpoint:** `PUT /tasks/:id` (example: `/tasks/1`)
- **Request body (JSON):** any of the following fields

{ "title": "Learn JavaScript", "description": "Practice REST APIs", "completed": true }

- **Success response (200):** the updated task

{ "id": 1, "title": "Learn JavaScript", "description": "Practice REST APIs", "completed": true }

- **Error responses:**
  - 404: `{ "message": "Task not found" }`
  - 400: `{ "message": "Task title is required" }` (if `title` is sent but empty or not a string)
- **Reasoning:** Users need to edit a task's text and mark it as done. The same endpoint handles both, so the API stays small. Fields that are not sent keep their old values, which prevents accidental data loss.

3.6 Delete a Task

- **Endpoint:** `DELETE /tasks/:id` (example: `/tasks/1`)
- **Request body:** None
- **Success response (200):**

{ "message": "Task deleted successfully" }

- **Error response (404):** `{ "message": "Task not found" }`
- **Reasoning:** Users must be able to remove tasks they no longer need. DELETE is the standard method for removal. Deleting the same ID twice returns 404 the second time, because the resource is already gone.

4. How the APIs Fulfil CRUD

CRUD Operation	HTTP Method	Endpoint	What it does
Create	POST	`/tasks`	Adds a new task
Read (all)	GET	`/tasks`	Lists every task

Read (one)	GET	`/tasks/:id`	Fetches a single task
Update	PUT	`/tasks/:id`	Edits details or toggles completion
Delete	DELETE	`/tasks/:id`	Removes a task

Together, these five endpoints cover the full lifecycle of a to-do item: a user creates a task, sees it in the list, opens it for details, updates it or marks it complete, and finally deletes it. The health check route supports development and monitoring.

5. RESTful Design Principles Followed

- **Resource-based URLs:** URLs use the noun `tasks`, not verbs like `/createTask` or `/deleteTask`. The action is carried by the HTTP method.
- **Correct HTTP methods:** GET reads, POST creates, PUT updates, DELETE removes.
- **Meaningful status codes:** 200 for success, 201 for created, 400 for bad input, 404 for a missing resource.
- **Stateless:** each request contains everything the server needs (ID in the URL, data in the JSON body). The server does not remember the client between requests.
- **JSON everywhere:** requests and responses use JSON, and a consistent `message` field is used for errors.
- **Consistent URL structure:** `/tasks` for the collection and `/tasks/:id` for a single item.

6. Potential Challenges

1. **Data lost on restart.** Tasks are stored in a plain in-memory array, so every server restart empties the list. A database such as MongoDB is needed for persistence.
2. **ID generation.** A simple counter (`nextId`) works for one server, but it resets on restart and could clash with multiple servers. Database-generated IDs solve this.

3. **Input validation.** Missing, empty, or wrongly typed fields (for example, a number as the title) can crash the server. Every route needs careful checks, and a validation library could help as the app grows.
4. **Invalid IDs.** A non-numeric ID such as `/tasks/abc` becomes `NaN`, which will simply return "not found". This should be handled with a clearer 400 response.
5. **PUT versus PATCH.** PUT strictly means replacing the whole resource, while partial updates are normally done with PATCH. Choosing the right method and documenting it avoids confusion.
6. **Error handling.** Unexpected errors should return a clean 500 response instead of crashing. A central error-handling middleware will be needed.
7. **Testing without a frontend.** APIs have to be tested with tools such as Postman or PowerShell, and repeated testing needs fresh data after each restart.
8. **CORS.** When the React frontend calls the API from a different port, CORS must be enabled on the server.
9. **Scalability and security.** As the app grows it will need pagination for large lists, authentication so each user sees only their own tasks, and protection against bad input.

7. Conclusion

The five core endpoints plus a health check give a complete, RESTful foundation for the To-Do List app. They are simple, follow standard HTTP conventions, and map directly onto Create, Read, Update, and Delete. The challenges listed above, mainly persistence, validation, and error handling, will be addressed in the implementation classes.